Currency Exchange Kiosk - Midterm Documentation

Name: Mac McCoy **Course:** CSC 1111 **Program:** `Midterm.cpp`

1. Name and Description

Currency Exchange Kiosk is a console program that simulates a multilingual currency exchange kiosk, such as a traveler might encounter in a European Shengen region airport. The user picks a language (English, German, romanized Arabic, or romaji Japanese), picks an input and output currency (USD, GBP, EUR, JPY, AED), and enters an amount. The program converts the amount to EUR, enforces a minimum of €50 equivalent, applies a tiered fee, converts to the output currency, and asks for confirmation before completing the exchange.

2. Inputs and Outputs

Inputs	Type	Notes
Language choice	int	1-4; anything else defaults to English
Input currency	int	1-5 (USD, GBP, EUR, JPY, AED). Input and output currency must be different; the same choice causes an error and restart at currency selection
Output currency	int	1-5. Must differ from the input currency
Currency confirmation	char	First character of the answer: Y/y continues, anything else restarts
Amount	double	Must be worth at least €50
Exchange confirmation	char	Y/y or a blank Enter completes, anything else cancels
Restart choice	char	Y/y starts a new exchange, anything else exits

Outputs	Notes
Language menu and currency menu	Language menu is shown in all four languages
Same-currency error	Shown when input and output currency are the same
Currency recap	Chosen input and output symbols
Minimum-amount error	Shown when the amount is under €50 equivalent
Summary	Amount entered, fee percent and EUR fee, total output
Real-world denomination rounding	The kiosk rounds all final output amounts to the nearest ¥5 for JPY, or the nearest 0.05 for USD, GBP, EUR, and AED
Success, thank-you, cancel, goodbye messages	All in the chosen language

3. How to Use the Program

1. Compile and run: `g++ -std=c++17 -o midterm Midterm.cpp && ./midterm`
2. Type the number of the language you can read and press Enter.
3. Type the number of the currency you have, then the number of the currency you want.
4. If you choose the same currency for input and output, the kiosk shows an error and asks you to choose again.
5. Review the recap and type `Y` to confirm (anything else restarts).
6. Enter the amount. If it is worth less than €50 you are asked again.
7. The kiosk rounds the final output to the nearest ¥5 (JPY) or 0.05 (all other currencies).
8. Review the fee and total, then type `Y` to confirm (anything else cancels).
9. Type `Y` to make another exchange, or anything else to exit.

4. Pseudocode

Each step letter matches a `// Step X:` comment in `Midterm.cpp`.

```
A. Repeat the kiosk transaction while kioskRunning is true
B.   Show the four-language welcome menu and read languageChoice
       If languageChoice is not 1-4, use English (index 0)
       Otherwise subtract 1 to get the array index
C.   Show the currency list
       Read inputCurrencyChoice and outputCurrencyChoice (subtract 1 from each)
D.   If inputCurrencyChoice == outputCurrencyChoice:
       Show an error message about identical currencies
       Restart at step C
```

```
E.   Show the recap: exchangeIntro + input symbol, receiveIntro + output symbol
F.   Ask to confirm the currency choices
       If not Y/y, go back to step B
G.   Repeat:
       Show "input symbol -> output symbol" and the amount prompt
       Read amountEntry
       If input currency is EUR, amountInEUR = amountEntry
       Otherwise amountInEUR = amountEntry * rateInputToEUR[input]
       If amountInEUR < minAmountEUR, show the too-low message
     Until amountInEUR >= minAmountEUR
H.   For each fee tier from lowest to highest:
       If amountInEUR >= feeTierThreshold[tier], exchangeFeePercent = feeTierPercent[tier]
I.   exchangeFeeAmount = amountInEUR * exchangeFeePercent / 100
     If output currency is EUR, outputAmount = amountInEUR - exchangeFeeAmount
     Otherwise outputAmount = (amountInEUR - exchangeFeeAmount) * rateEURToOutput[output]
     Round for real-world denominations:
       If output currency is JPY, outputAmount = round(outputAmount / 5.0) * 5.0
       Otherwise outputAmount = round(outputAmount / 0.05) * 0.05
J.   Show amount entered, fee percent with EUR fee, and rounded total output
K.   Ask to confirm the exchange
       If not Y/y, show the cancelled message and go back to step B
L.   Show the success and thank-you messages
M.   Ask whether to start another exchange
       If not Y/y, show goodbye and set kioskRunning to false
```

Fee tiers use the amount in EUR. A tier applies when the amount is at or above its threshold. The highest threshold reached determines the fee percent.

EUR value	Fee %
50-99.99	6.0
100-199.99	5.0
200-499.99	4.0
500-999.99	3.0
1000-1999.99	2.0
2000 and up	1.5

5. Test Data

Sample rates (A real-world program would API call and check the current accepted rate every day before start of business.):

Currency	To EUR	From EUR
USD	0.92	1.09
GBP	1.17	0.855
EUR	1.0	1.0
JPY	0.0061	164.0
AED	0.25	3.97

Menu order: 1 USD, 2 GBP, 3 EUR, 4 JPY, 5 AED. All rows were verified by running the program with scripted input.

B. Language selection (step B)

Input	Expected
1	English prompts
2	German prompts
3	Romanized Arabic prompts
4	Romaji Japanese prompts
99, 0, -1	Defaults to English
blank Enter	Not testable: `cin >>` skips blank lines and waits

C. Currency selection (step C)

Input	Output	Expected
USD	EUR	Valid
EUR	USD	Valid
JPY	GBP	Valid
AED	JPY	Valid

D. Same-currency check (step D)

Input	Output	Expected
EUR	EUR	Error message, restart at currency selection
USD	USD	Error message, restart at currency selection
USD	EUR	No error, continue to recap

F. Currency confirmation (step F)

Input	Expected
Y / y	Continue to amount
N	Restart at language menu
abc	Restart at language menu

G. Minimum amount (step G, EUR equivalent must be at least 50)

Currency	Amount	EUR value	Expected
USD	40	36.80	Reject, ask again
GBP	40	46.80	Reject
EUR	49	49.00	Reject
JPY	5000	30.50	Reject
AED	200	50.00	Accept
USD	60	55.20	Accept
GBP	50	58.50	Accept
EUR	50	50.00	Accept
JPY	10000	61.00	Accept

H. Fee tiers (step H, EUR input to USD output; fee % and EUR fee shown)

EUR value	Fee %	Fee amount (EUR)
50	6.0	3.00
100	5.0	5.00
200	4.0	8.00
500	3.0	15.00
1000	2.0	20.00
2000	1.5	30.00

I. Full conversions (step I, fee deducted in EUR, then converted)

Input	Output	EUR value	Fee	Raw output	Expected (rounded)
100 USD	GBP	92.00	6% = 5.52	73.9404	73.95
100 GBP	JPY	117.00	5% = 5.85	18228.6	18230
100 EUR	AED	100.00	5% = 5.00	377.15	377.15
10000 JPY	USD	61.00	6% = 3.66	62.5006	62.50
60 USD (German)	GBP	55.20	6% = 3.312	44.3642	44.35

I2. Rounding (real-world denominations) The kiosk cannot dispense sub-denomination currency, so the output is rounded to the nearest ¥5 (JPY) or 0.05 (USD, GBP, EUR, AED).

Output currency	Raw output	Expected rounded
USD	44.3642	44.35
GBP	73.9404	73.95
EUR	377.153	377.15
AED	62.5006	62.50
JPY	18228.6	18230

J. Summary display (step J)

Shows, in the chosen language:

- input amount with its symbol
- fee percent and the EUR fee
- final output amount with its symbol

K. Exchange confirmation (step K)

Input	Expected
Y / y	Success and thank-you messages
N	"Transaction cancelled", restart at language menu

M. Restart (step M)

Input	Expected
Y / y	Restart at language menu
N / n	Goodbye, program exits

6. Lessons Learned

- Checking the program against the rubric showed that it needed `char` and `switch`, so the Y/N answers became `char` variables with `switch` decisions.
- Testing every branch exposed problems that were easy to miss: bad or non-numeric input, identical input and output currencies, inconsistent fee tiers, and unrounded money.
- Real-world limits matter: the kiosk cannot dispense coins smaller than ¥5 or 0.05, so outputs are rounded.
- Writing the pseudocode first (steps A-M) and matching it to the code comments made the program much easier to debug.

7. AI Disclosure

- I do not speak German, Arabic, or Japanese. The translations in the prompts are automated, so they may contain mistakes or sound unnatural to a native speaker.
- I used GitHub Copilot to clean up formatting and to help me debug the project.
- Similar currency kiosk projects may exist, but the idea for this program, implemented my way, was my own.